

Open-SDD: Arquitectura Ejecutable, Harnessing Zero-Trust y Living Specifications para la Ingeniería de Software Agéntica Enterprise

Mario Alejandro Ramos* Open-SDD Research & Architecture Initiative

Septiembre de 2026 (Preprint)

Resumen

A medida que la inteligencia artificial generativa transiciona aceleradamente desde el autocompletado asistido hacia enjambres autónomos multi-agente (*multi-agent autonomous swarms*), la ingeniería de software corporativa y de producción masiva se enfrenta a una crisis de gobernanza estructural: la capacidad generativa estocástica de los modelos de lenguaje supera ampliamente los mecanismos deterministas de verificación disponibles. Los pipelines agénticos no controlados disparan el *write throughput*, degradando severamente la cohesión sistémica, cuadruplicando la duplicación de código y acumulando deuda técnica silenciosa.

Este artículo presenta **Open-SDD**, un marco metodológico y *reference harness* abierto, agnóstico al modelo y gobernado por principios de *Zero-Trust*. Fundamentado en la quinta generación de abstracción de software—donde la intención arquitectónica se convierte en un plano de control ejecutable (*SpecOps*) promovido por la literatura reciente de la Oficina del CTO de Red Hat—Open-SDD formaliza un lazo cibernético cerrado de cuatro etapas: *propose* → *act* → *verify* → *learn*. Mediante la unificación de protocolos en estándares **sdd-*** y la persistencia de especificaciones vivas en **.sdd/**, Open-SDD introduce seis innovaciones fundamentales: (i) una **Tríada Documental** gobernada por sintaxis formal EARS y proveniencia inmutable en Git; (ii) la disección y superación de los ****Ocho Antipatrones Críticos de la Adopción Agéntica**** documentados en la industria a septiembre de 2026, destacando la erradicación de la falacia de personificación agéntica (*Role-Based Agent Trap*) en favor de clases de capacidad funcional; (iii) un modo estricto de control de versiones (**Strict Git Mode**) con automatización *SpecOps* y compuerta inmutable **Gate 0**, bloqueando de forma absoluta cualquier generación de código sin especificaciones aprobadas formalmente por supervisión humana; (iv) un ****Motor de Decisión Contextual Adaptativo**** (PASS/WARN/ASK/BLOCK) que mitiga la *Bypass Conjecture* ($E_{\text{ef}} = E_{\text{nom}}(1 - d(\phi))$) para armonizar el avance veloz del desarrollador (*Flow State*) con controles de seguridad sin fricción; (v) un sustrato de validación continua basado en ****Agentic Quality Engineering (Agentic QE)**** bajo la metodología PACTS (Proactive, Autonomous, Collaborative, Targeted, Structured) desplegado sobre flotas Model Context Protocol (MCP) e invariantes metamórficos; y (vi) un ****Espectro Universal de Modernización Brownfield**** que cubre más del 90 % del desarrollo de software real, escalando adaptativamente desde desarrolladores individuales y *startups* ágiles hasta arquitecturas *hyperscaler* (AWS, GCP, Azure) mediante ingeniería inversa *code-first* (**/sdd-getspecs**), congelamiento de comportamiento observable (*Golden Characterization Locking* de Michael Feathers) y desacoplamiento progresivo (*Strangler Fig Pattern* de Martin Fowler). Demostramos cómo Open-SDD restituye la soberanía humana sobre la intención, la ética y las fronteras de diseño, transformando el código fuente en un activo efímero y reproducible bajo demanda (*Ambient Code*).

Palabras clave: Spec-Driven Development, Agentic Software Engineering, Zero-Trust Architecture, Living Specifications, Modernización Brownfield Universal, Agentic QE, PACTS, Strict Git Mode, Bypass Conjecture, Antipatrones Agénticos, Gobernanza de IA.

*Ingeniero en Computación e Informática, especialista en Spatial Computing y Arquitecturas Agénticas, NTT DATA. Email: mario.ramos@nttdata.com

1. Introducción y Estado del Arte a Septiembre de 2026

Tras más de tres años de integración generalizada de Large Language Models (LLMs) en el ciclo de vida del software (SDLC), la industria tecnológica ha alcanzado un punto de inflexión empírico: la divergencia radical entre la aceleración subjetiva percibida por los ingenieros y la degradación objetiva de la estabilidad sistémica de entrega.

Ensayos de control aleatorizado conducidos por METR demostraron que desarrolladores altamente experimentados operando asistentes de IA sin restricciones de arnés resultaron hasta un 19% más lentos en bases de código maduras, pese a reportar una percepción de mejora de velocidad del 20% [1]. A nivel estructural, evaluaciones longitudinales de código fuente sobre 211 millones de líneas realizadas por GitClear evidenciaron una cuadruplicación en la proliferación de bloques de código duplicados (*code clones*) y un desplome de la refactorización a menos del 10% del volumen total de cambios [2]. De manera convergente, la telemetría global de Google Cloud DORA reveló que por cada 25 puntos porcentuales de incremento en adopción no gobernada de asistentes agénticos, la estabilidad de entrega del sistema sufrió un deterioro medible del 7.2% [3].

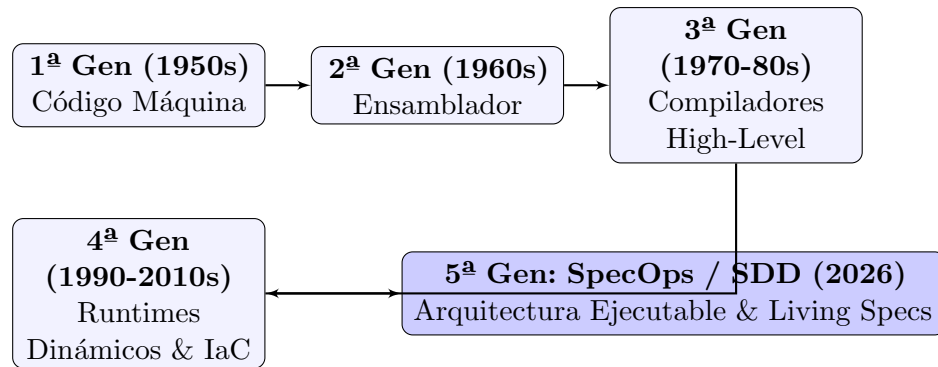


Figura 1: La evolución de la abstracción del software: De la manipulación binaria a la arquitectura ejecutable (adaptado de Griffin & Carroll, Red Hat Office of the CTO [4]).

El diagnóstico formal es contundente: **el cuello de botella en la ingeniería agéntica de 2026 no radica en la potencia generativa de los modelos, sino en la ausencia de un plano de control determinista sobre un proceso estocástico**. Cuando los agentes operan sin fronteras arquitectónicas explícitas, inducen acoplamiento de estado inadvertido, colapsan la atomicidad transaccional y sufren de sesgo de auto-evaluación (*self-review bias*) al generar y validar código dentro del mismo contexto cognitivo.

Como plantean Leigh Griffin y Ray Carroll desde la Oficina del CTO de Red Hat [4], la ingeniería de software ha inaugurado su ****Quinta Generación de Abstracción**** (Figura 1). En este paradigma (*Spec-Driven Development*), la arquitectura deja de ser documentación pasiva archivada en wikis obsoletas y se convierte en el plano de control ejecutable (*SpecOps*).

Cuadro 1: Inversión Arquitectónica: Modelo Clásico vs. Modelo Open-SDD (adaptado de [4]).

Dimensión	Modelo Clásico (Legacy)	Modelo Open-SDD (SpecOps)
Fuente Canónica de Verdad	Código fuente productivo	Especificaciones ejecutables vivas en Git
Rol de la Arquitectura	Consultivo (diagramas pasivos en wikis)	Ejecutable, vinculante y verificado en compuerta
Naturaleza del Código	Activo primario de inversión a largo plazo	<i>Ambient Code</i> : desechable, reproducible bajo demanda
Detección de <i>Drift</i>	Forense post-facto (incidentes en producción)	Preventiva pre-runtime mediante compuertas continuas
Comportamiento en Runtime	Emergente, estocástico e impredecible	Arquitectónicamente determinista
Supervisión Humana	Revisión manual línea por línea de diffs	Gobernanza de alto nivel: intención, ética y límites

Esta inversión arquitectónica (Tabla 1) redefine el valor del código: el código se vuelve *ambiental y efímero*, mientras que la especificación viva estructurada en Git es la verdad inmutable del sistema.

2. Taxonomía de Antipatrones en la Adopción Agéntica (Ramos 2026)

Basado en la evidencia de fallas sistémicas observadas en despliegues corporativos reales durante 2025 y 2026 documentadas por Ramos [5], formulamos una taxonomía de **ocho antipatrones críticos de adopción agéntica** que Open-SDD neutraliza de manera programática:

Cuadro 2: Taxonomía de Antipatrones Agénticos y Contramedidas de Open-SDD (basado en Ramos [5]).

Antipatrón Agéntico	Manifestación y Riesgo Operacional	Patrón Correctivo Open-SDD
1. Role Personification	Asignar "roles teatrales" ("Eres un Arquitecto", "Eres un QA Lead") a los agentes. Provoca <i>prompt bloat</i> , colusión algorítmica y alucinaciones jerárquicas.	Clases de Capacidad Funcional: Los agentes son ejecutores efímeros acotados por herramientas, presupuesto de tokens, temperatura y frontera (<code>_Boundary: .</code>).
2. Context Dumping	Volcar repositorios enteros a la ventana de contexto. Provoca <i>Lost-in-the-Middle</i> [13], envenenamiento y alucinación de APIs.	Graph before Grep: Extracción determinista de AST vía Tree-Sitter/LSP hacia <code>.sdd/memory/graph/</code> . Solo se inyecta el grafo de dependencias relevante.
3. Binary Gate Paralysis	Compuertas rígidas que bloquean indistintamente acciones reales de meras representaciones (ej. detectar <code>rm -rf</code> en un test).	Motor de Decisión Contextual: Clasificación graduada en 4 niveles (PASS/WARN/ASK/BLOCK) evaluando superficie y contexto.
4. Escape-Hatch Proliferation	Multiplicación de flags de bypass (<code>--force, BYPASS=1</code>) por fatiga de falsos positivos, colapsando la seguridad ($E_{ef} \rightarrow 0$).	Receipts Criptográficos Unívocos: El bypass requiere aprobación HITL con hash de un solo uso registrado en el log de auditoría.
5. Checkbox Closure	Marcar tareas como completadas verificando presencia de cadenas o checkboxes, sin validar el objetivo real.	Verificación Empírica Obligatoria: Tarea cerrada exclusivamente con código de salida cero, tests en verde y captura fresca de stdout.
6. Unbounded Retries	Loops de reintento descontrolados ante fallas de ejecución agéntica, colapsando APIs y agotando cuotas.	Presupuesto de Reintentos & Circuit Breaker: Máximo de reintentos acotados, <i>exponential backoff with jitter</i> y retroalimentación en <code>tasks.md</code> .
7. Asymmetric Safeguards	Controles estrictos en entornos productivos pero nulos en desarrollo o evaluación local agéntica.	Zero-Trust de Extremo a Extremo: Idénticas compuertas de seguridad ejecutadas en local, sandbox MCP y pipelines de CI/CD.
8. Decorative Metrics	Paneles de monitoreo con métricas cosméticas no fundamentadas en comandos de medición comprobables.	Telemetría Determinista PACTS: Cada indicador en <code>sdd-audit</code> enlaza al comando que lo genera y al hash del commit Git correspondiente.

2.1. Desmitificación del Rol: De la Personificación a las Clases de Capacidad

Un error capital identificado en las arquitecturas agénticas de primera generación consiste en emular la estructura organizativa humana mediante *system prompts* cargados de psicología artificial ("Piensa como un Senior DevOps", "Actúa como un Security Lead"). Los modelos de lenguaje carecen de autoconciencia deontológica; asignarles una personalidad introduce verbosidad teatral, sesgos de complacencia (*sycophancy*) y alucinación de autoridad.

Open-SDD erradica este enfoque: **los agentes no son personas; son unidades de ejecución atómicas acotadas por un contrato de capacidad:**

$$\text{AgentExecutionContract} = \langle \mathcal{M}_{\text{class}}, \mathcal{T}_{\text{tools}}, \Theta_{\text{temp}}, \mathcal{B}_{\text{boundary}}, \mathcal{C}_{\text{criteria}} \rangle \quad (1)$$

donde $\mathcal{M}_{\text{class}}$ es la clase de razonamiento del modelo (Frontier Reasoning, Code-Specialized o Economy), $\mathcal{T}_{\text{tools}}$ es el set de herramientas de menor privilegio expuestas vía MCP, Θ_{temp} es la temperatura vinculada (con $\Theta = 0$ para auditorías e invariantes), $\mathcal{B}_{\text{boundary}}$ es la lista cerrada de rutas de archivos autorizados, y $\mathcal{C}_{\text{criteria}}$ son los criterios de aceptación EARS a verificar.

3. Arquitectura del Sistema: El Lazo Cibernético y el Equilibrio de Flujo

Open-SDD estructura la interacción agéntica en un lazo cibernético cerrado (Figura 2):

$$\text{Loop} = \langle \text{Propose} \longrightarrow \text{Act} \longrightarrow \text{Verify} \longrightarrow \text{Learn} \rangle \quad (2)$$

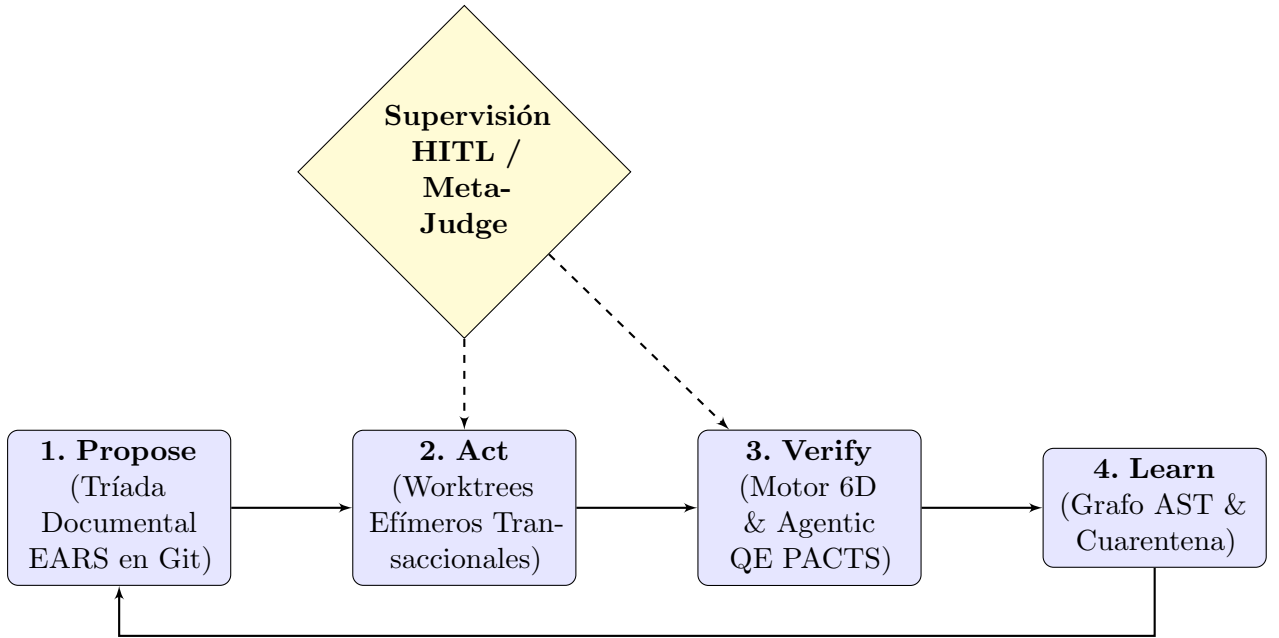


Figura 2: Lazo de control cibernético de Open-SDD con supervisión humana y arbitraje determinista.

3.1. El Equilibrio entre Gobernanza y Velocidad (*Flow State*)

El diseño de Open-SDD responde a una máxima de usabilidad crítica: **la gobernanza que destruye la velocidad del desarrollador termina siendo sabotada**. Si el harness exige múltiples aprobaciones manuales para un cambio trivial, el desarrollador buscará atajos de evasión.

Para preservar el estado de flujo (*Flow State*) sin sacrificar determinismo, Open-SDD introduce un **Mecanismo de Aprobación Graduada**:

- **Flujo Fast-Track** (-y, --auto): Para tareas atómicas confinadas a módulos de bajo riesgo y radio de impacto nulo, el sistema avanza automáticamente entre compuertas siempre que los tests unitarios y validaciones de sintaxis se mantengan en verde.

- **Flujo Riguroso con Revisión Humana Obligatoria:** Reservado para inicialización de especificaciones, cambios en contratos de interfaz pública o mutaciones en componentes de alto impacto (*Core Crítico*).

3.2. Fase 1: Propose (La Tríada Documental)

Todo incremento funcional exige una especificación sincrónica almacenada bajo el árbol de especificaciones vivas `.sdd/specs/<feature>/`:

1. **requirements.md:** Criterios de aceptación estructurados en sintaxis **EARS** (*Easy Approach to Requirements Syntax*) [7]. EARS erradica la ambigüedad del lenguaje natural mediante 5 patrones matemáticamente analizables (Ubiquitous, Event-driven, State-driven, Optional features, Unwanted behavior).
2. **design.md:** Topología de componentes, diagramas Mermaid interactivos, contratos OpenAPI/GraphQL y Decisiones Arquitectónicas Registradas (**ADRs**).
3. **tasks.md:** Grafo Acíclico Dirigido (DAG) que serializa el trabajo en tareas atómicas TDD (RED $\rightarrow$ GREEN) anotadas con `_Depends:_` y `_Boundary:_`.

3.3. Fase 2: Act (Worktrees Transaccionales y Aislamiento de Estado)

Para evitar condiciones de carrera en el sistema de archivos, cada tarea se ejecuta en un *Git Worktree* aislado sobre una rama efímera vinculada. Los subagentes no comparten directorio de trabajo en sucio. Si una tarea genera regresiones o no satisface sus aserciones, su worktree se descarta; el estado del repositorio principal jamás sufre contaminación.

3.4. Fase 3: Verify (La Conjetura del Bypass y el Motor Contextual 6D)

Tal como modeló formalmente Ramos [5], la efectividad real de la seguridad agéntica está condicionada por la tasa de falsos positivos:

$$E_{\text{ef}}(\phi) = E_{\text{nom}}(\phi) \cdot (1 - d(\phi)) \quad (3)$$

donde $d(\phi)$ es la probabilidad de desactivación humana provocada por fricción y fatiga.

El **Motor de Decisión Contextual** de Open-SDD evalúa seis dimensiones en tiempo real (Fase del ciclo de vida, Entorno de ejecución, Superficie de llamada, Tipo de artefacto, Confianza del modelo y Matriz de riesgo) para emitir cuatro veredictos deterministas:

- **PASS:** Acciones seguras ejecutadas en fixtures o documentación.
- **WARN:** Operaciones con desvío no crítico registradas en el log sin detener la ejecución.
- **ASK:** Ambigüedad de frontera que exige autorización HITL mediante token criptográfico efímero.
- **BLOCK:** Parada de emergencia ante riesgos comprobados (comandos destructivos en producción, fugas de secretos).

3.5. Fase 4: Learn (Malla de Memoria y Cuarentena Anti-Envenenamiento)

El conocimiento operativo se almacena de forma particionada:

- **Memoria de Sesión** (`.sdd/memory/session/`): Bitácoras temporales compactadas para mitigar el consumo de contexto.

- **Grafo AST** (`.sdd/memory/graph/`): Grafo determinista de símbolos y llamadas (*Graph before Grep*).
- **Cuarentena de Aprendizaje**: Cualquier patrón detectado tras un fallo debe pasar por un validador a temperatura cero para evitar ataques de inyección indirecta antes de consolidarse en `.sdd/steering/`.

4. Strict Git Mode y el Ciclo de Vida SpecOps: Spec-as-Code

Inspirado en los principios de automatización de CI/CD para especificaciones detallados en la literatura de Red Hat [4], Open-SDD implementa **Strict Git Mode** (`.sdd/settings/git.json`), convirtiendo a Git en el árbitro único del ciclo agéntico.

$$\text{State}_{\text{Git}} = \langle \text{Init Seed} \longrightarrow \text{Feature Branch} \longrightarrow \text{Spec Lock} \longrightarrow \text{Gated Impl} \longrightarrow \text{Verified Push} \rangle \quad (4)$$

4.1. El Invariante Gate 0

Open-SDD prohíbe de forma matemática que un agente toque código fuente sin una especificación aprobada por un humano:

$$\text{Gate}_0(F) = \begin{cases} \text{AUTHORIZED}, & \text{si } \text{spec.json.phase} = \text{"approved"} \wedge \text{Approvals}(F) = \text{True} \\ \text{HALTED}, & \text{en caso contrario (Retorna E_SPEC_UNAPPROVED)} \end{cases} \quad (5)$$

Cualquier intento de ejecutar `/sdd-impl` en una especificación en borrador o inicializada aborta de inmediato.

4.2. La Tríada de Invariantes Críticos y Gobernanza Dual

Para optimizar el flujo de entrega continua y garantizar la adopción práctica de la arquitectura agéntica mitigando la *Bypass Conjecture* formulada en la Ecuación ??, Open-SDD estructura su marco de verificación a través de la **Tríada de Invariantes Críticos**:

1. **Invariante de Frontera y Blast Radius** ($\mathcal{G}_1$): Confinamiento estricto a las rutas declaradas en `_Boundary:-`, impidiendo la mutación descontrolada de archivos no autorizados.
2. **Invariante de Contrato Mínimo** ($\mathcal{G}_2$): Existencia de especificaciones mínimas (criterios EARS) para impedir la invención de alcance alucinado.
3. **Invariante de Evidencia Empírica Fresca** ($\mathcal{G}_3$): Ejecución determinista de pruebas unitarias e invariantes metamórficos sin regresiones.

Para balancear agilidad y conformidad, Open-SDD introduce una **Gobernanza Dual** configurable vía `.sdd/settings/governance.json`:

- **Modo Libre** (`fluid`, **por defecto**): Maximiza la velocidad del flujo de desarrollo (*flow state*) en startups y equipos ágiles. Las compuertas secundarias actúan como advertencias no bloqueantes (*non-blocking telemetry*), habilitando el avance en vía rápida (*fast-track* con `-y` y `--auto`) sin pausas burocráticas.
- **Modo Soberano** (`strict`): Diseñado para industrias reguladas y entornos hyperscaler. Activa el bloqueo duro de Gate 0, registro inmutable en Git y auditorías formales conforme al EU AI Act y NIST AI RMF.

4.3. Automatización SpecOps

Al aprobarse la Tríada Documental, Open-SDD automatiza la orquestación en Git:

1. **Branch Isolation:** Creación automática de la rama de trabajo `feat/` o `Slug(F)`.
2. **Hito Spec Lock:** Commit formal de las especificaciones aprobadas (`spec(slug): lock approved documentary triad`) y push a remoto.
3. **Verified Push & PR:** Tras validar la implementación con tests en verde y Agentic QE (`/sdd-validate-impl`), Open-SDD publica la rama y genera automáticamente el Pull Request documentando la trazabilidad completa.

5. Validación Continua: Autonomous Quality Engineering (Agentic QE) y PACTS

Para responder a la alta velocidad de generación multi-agente, Open-SDD integra de forma nativa el marco de **Agentic QE** [9], gobernado por la metodología **PACTS**:

- **Proactive:** Modelado de riesgos de regresión antes de confirmar cambios.
- **Autonomous:** Generación automática de pruebas metamórficas e invariantes de propiedades sin requerir scripting manual:

$$\forall x, x' \in \mathcal{D}, \quad R(x, x') \implies R(\mathcal{F}_{\text{legacy}}(x), \mathcal{F}_{\text{modernized}}(x')) \quad (6)$$

- **Collaborative:** Auditoría cruzada donde subagentes revisores independientes desafían el código generado (`sdd-review`) erradicando el sesgo de auto-revisión.
- **Targeted:** Confinamiento estricto al radio de impacto (`Boundary(Ti)`).
- **Structured:** Emisión de evidencia empírica estructurada hacia el informe de auditoría (`sdd-audit`).

6. Mecanización de las Directrices de Karpathy

Principios pragmáticos formulados por Andrej Karpathy [6] son transformados en invariantes operacionales de compilación:

Cuadro 3: Mecanización de las Directrices de Karpathy en Open-SDD.

Directriz Karpathy	Mecanismo de Arrastre	Invariante Operacional en Open-SDD
Think Before Coding	Fase de Descubrimiento (<code>sdd-discovery</code>)	Prohíbe supuestos silenciosos; obliga a clarificación interactiva previa a la spec.
Simplicity First	Hard Gate de Complejidad	Rechaza abstracciones prematuras, configurabilidad innecesaria y diffs sobredimensionados.
Surgical Changes	Compuerta de Frontera (<code>_Boundary:_</code>)	Restringe escrituras únicamente a los archivos declarados; prohíbe formateos colaterales.
Goal-Driven Execution	Compuerta de Verificación (<code>sdd-verify</code>)	Prohíbe el cierre de tareas sin código de salida cero, aserciones en verde y evidencia capturada.

7. Modernización Brownfield Universal: Del Desarrollador Solo al Hyperscaler

En el mundo real, ****más del 90 % del software existente es Brownfield****. La creencia de que SDD solo es aplicable a desarrollos nuevos o a grandes corporaciones es un mito infundado. Open-SDD propone un ****espectro progresivo en 4 niveles****:

Cuadro 4: El Espectro Brownfield Universal de Open-SDD según Escala Organizacional.

Dimensión	Tier 1: Solo / Freelance	Tier 2: Startup / SMB	Tier 3: Scale-Up / OSS	Tier 4: Enterprise / Hyperscaler
Objetivo Primario	Grounding y cero amnesia	Memoria viva, sin deuda	Desacople de fronteras	Zero-Trust y cumplimiento legal
Punto de Entrada	/sdd-getspecs (rápido)	/sdd-getspecs (por módulo)	/sdd-getspecs + Gap	Grafo AST + Matriz Blast Radius
Foco de Pruebas	Tests unitarios existentes	Unitarios + Integración en PR	Pruebas de frontera acotadas	Agentic QE (Invariantes PACTS)
Compuerta Aprobación	Fast-track (-y, --auto)	Revisión por pares	Aprobación en Git vía PR	Strict Git Mode + Gate 0 Humano
Cumplimiento	Mínimo (sin fricción)	Registro en commits/PRs	Trazabilidad de ADRs	sdd-audit (EU AI Act, NIST)
Patrón Migración	Refactorización quirúrgica	Feature flags simples	Proxy Strangler Fig	Multi-Cloud Canary / Mirroring

- **Tier 1: Desarrollador Solo & Freelancer (*Frictionless Brownfield*)**: Al retomar proyectos tras meses de pausa, /sdd-getspecs extrae la realidad del código hacia .sdd/steering/, evitando que el asistente alucine librerías inexistentes o destruya patrones establecidos.
- **Tier 2: Startups & Equipos Pequeños (*Anti-Deuda Técnica*)**: Erradica el conocimiento tribal (*bus factor* = 1). Genera un mapa de ruta vivo en steering/roadmap.md, acelerando el onboarding de nuevos desarrolladores de semanas a minutos.
- **Tier 3: Scale-Ups & Open Source (*Modernización Modular*)**: Aplica la herramienta /sdd-validate-gap para calcular radios de impacto y desacoplar monolitos en módulos atómicos con fronteras confinadas (_Boundary:_).
- **Tier 4: Enterprise & Hyperscalers (*Modernización Zero-Trust Regulada*)**: Activa el protocolo completo de 5 capas:
 1. **Capa 1: Reconocimiento Estructural (*Graph before Grep*)**: Mapeo AST estático de dependencias.
 2. **Capa 2: Congelamiento de Caracterización (*Golden Characterization Locking*)**: Basado en Michael Feathers [11], congela el comportamiento observable B_0 en Git antes de alterar código.
 3. **Capa 3: Semillas de Especificación Viva y Compuerta Humana**: Validación y firma de arquitectos previa a la implementación.
 4. **Capa 4: Implementación Quirúrgica (*Strangler Fig Pattern*)**: Basado en Martin Fowler [12], enruta tráfico progresivo mediante proxies y feature flags.
 5. **Capa 5: Auditoría y Cumplimiento**: Enlace de código a requerimientos, ADRs y tests vía sdd-audit.

7.1. Patrones de Nube en Hyperscalers

Open-SDD se integra con mecanismos nativos de nube: en **AWS** vía ALB Weighted Routing y VPC Traffic Mirroring para validación en modo sombra; en **Google Cloud** mediante Cloud Run y Traffic Director para canarios incrementales (1 % → 10 % → 100 %); y en **Microsoft Azure** con Azure Front Door y Azure Chaos Studio para resiliencia operacional.

8. Auditabilidad, Trazabilidad y Cumplimiento Regulatorio

Open-SDD automatiza la generación de evidencia de cumplimiento legal y normativo mediante el comando `sdd-audit`:

- **EU AI Act (Reglamento de IA de la UE)**: Documentación técnica exhaustiva (Art. 11), registro inmutable de sesiones de IA (Art. 12) y gobernanza con supervisión humana demostrable (Art. 14).
- **NIST AI RMF e ISO/IEC 42001**: Mapeo continuo de riesgos e inventario de controles verificables.
- **Genealogía de Decisiones**: Cada línea de código productivo enlaza bidireccionalmente a su requerimiento EARS, ADR fundacional, hash del modelo generador y firma del revisor humano.

9. Desafíos Abiertos y Fronteras de Investigación Futura

Declaramos formalmente cuatro áreas de investigación abierta para la evolución de la disciplina:

1. **Verificación Formal Dinámica con SMT Solvers (Z3 / Lean 4)**: Integración de demostradores formales de teoremas para probar matemáticamente la equivalencia semántica entre código legacy y código ambiental generado.
2. **Federación Descentralizada de Specs Multi-Repositorio**: Sincronización de contratos de especificación viva a través de flotas de microservicios y microfrontends sin crear cuellos de botella centralizados.
3. **Optimización Dinámica de Economía de Tokens**: Algoritmos de subasta en tiempo real para asignar presupuestos de tokens según el índice de criticidad y blast radius del componente.
4. **Firmas Criptográficas Post-Cuánticas en Spec Locks**: Incorporación de sellado criptográfico resistente a computación cuántica para compras públicas de defensa y banca central.

10. Conclusión

Open-SDD demuestra que la ingeniería agéntica de alta velocidad a escala no requiere modelos más grandes, sino un arnés de control determinista, riguroso y libre de fricción. Al tratar las especificaciones como código vivo en Git, erradicar los antipatrones de personificación, universalizar el brownfield y acoplar validación continua con Agentic QE, Open-SDD ofrece el estándar definitivo para transformar la generación estocástica en ingeniería de software predecible, segura y de calidad enterprise.

Referencias

- [1] J. Becker, N. Rush, E. Barnes, y D. Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. *arXiv preprint arXiv:2507.09089*, 2025.
- [2] GitClear. AI Copilot code quality: 2025 data suggests 4x growth in code clones. *GitClear Research Reports*, 2025.

- [3] Google Cloud DORA. *Accelerate State of DevOps Report 2024–2025: AI Adoption and System Stability*. Google, 2025.
- [4] L. Griffin y R. Carroll. Spec driven development: When architecture becomes executable. *InfoQ Architecture Series*, Enero 2026.
- [5] M. A. Ramos. SDD-First multi-agent orchestration for enterprise development: A zero-trust reference architecture for governing agentic software engineering at scale. *Preprint DOI: 10.13140/RG.2.2.29185.42087*, Agosto 2026.
- [6] A. Karpathy. Guidelines for mitigating common LLM coding pitfalls. Codificado en <https://github.com/multica-ai/andrey-karpathy-skills>, 2025–2026.
- [7] A. Mavin, P. Wilkinson, A. Harwood, y M. Novak. Easy approach to requirements syntax (EARS). En *Proc. 17th IEEE International Requirements Engineering Conference*, 2009.
- [8] L. Sampaio. *AI-Assisted SDD: Spec-Driven Development with Gemini, Claude, and ai-sdd*. Autoeditado, 2025.
- [9] D. Spiridonov. Agentic QE Framework: Bridging classical testing to autonomous quality engineering. *Quantum QE & Agentic-QE.dev*, 2025–2026. <https://agentic-qe.dev>.
- [10] R. Cohen. The PACT principles for autonomous agent governance. *Agentics Foundation Whitepapers*, 2024–2025.
- [11] M. Feathers. *Working Effectively with Legacy Code*. Prentice Hall, 2004.
- [12] M. Fowler. Strangler Fig application. <https://martinfowler.com/bliki/StranglerFigApplication.html>, 2004–2020.
- [13] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, y P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [15] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (UE) 2024/1689 por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial). *Diario Oficial de la Unión Europea*, L 2024/1689, Julio 2024.
- [16] National Institute of Standards and Technology (NIST). Artificial intelligence risk management framework (AI RMF 1.0). *NIST Trustworthy and Responsible AI*, NIST SP 1270, Enero 2023.